

Project Overview (DoDay)

[Goal: Labor Costs reduced by 67% in DoDay]

Labor Intensive

Labor Innovation

High Error Rate

Traditional Dessert Store

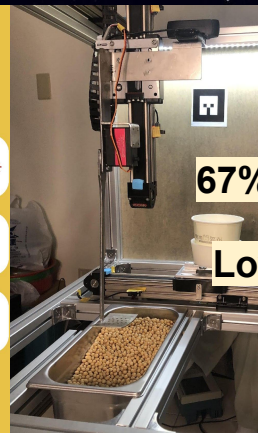
A typical store usually requires
3-4 people. Labor costs can be up
to 40% of the total cost



First automated cafe using
intelligent systems and robotics.
Only need **1 person** to manage.

67% Less Labor

Low Error Rate





Auto Dessert Maker



DoDay Ordering



DoDay Deliver

portal.do-day.com/doing/003-do-day-management-console/Manage/attendance

豆日子後台

Home / 豆日子後台 / 簽到打卡

簽到打卡

1) 打卡頁面
請點擊[此處](#)取得待簽卡人打卡專用二維碼。

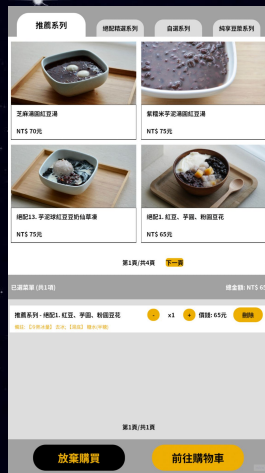
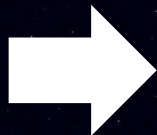
2) 簽到紀錄

打卡紀錄					
日期	工作時間	上班時間	下班時間	工作時數(h)	日累計時數(h)
2026-03-01	-	-	-	0	0
2026-03-02	-	-	-	0	0
2026-03-03	-	-	-	0	0
2026-03-04	-	-	-	0	0
2026-03-05	-	-	-	0	0
2026-03-06	-	-	-	0	0
2026-03-30	-	-	-	0	0
2026-03-31	-	-	-	0	0

DoDay Management



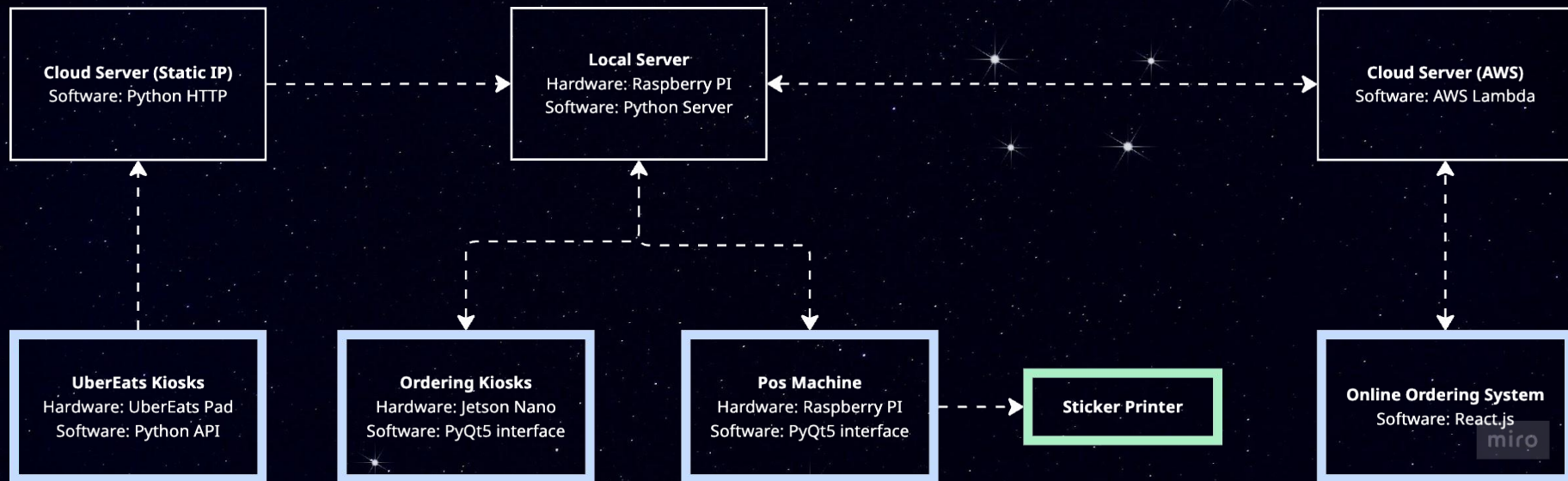
Design & Built DoDay



Problem: Staff Shortage + Complex Orders = Data Errors. To fix this, we must eliminate manual data entry entirely.

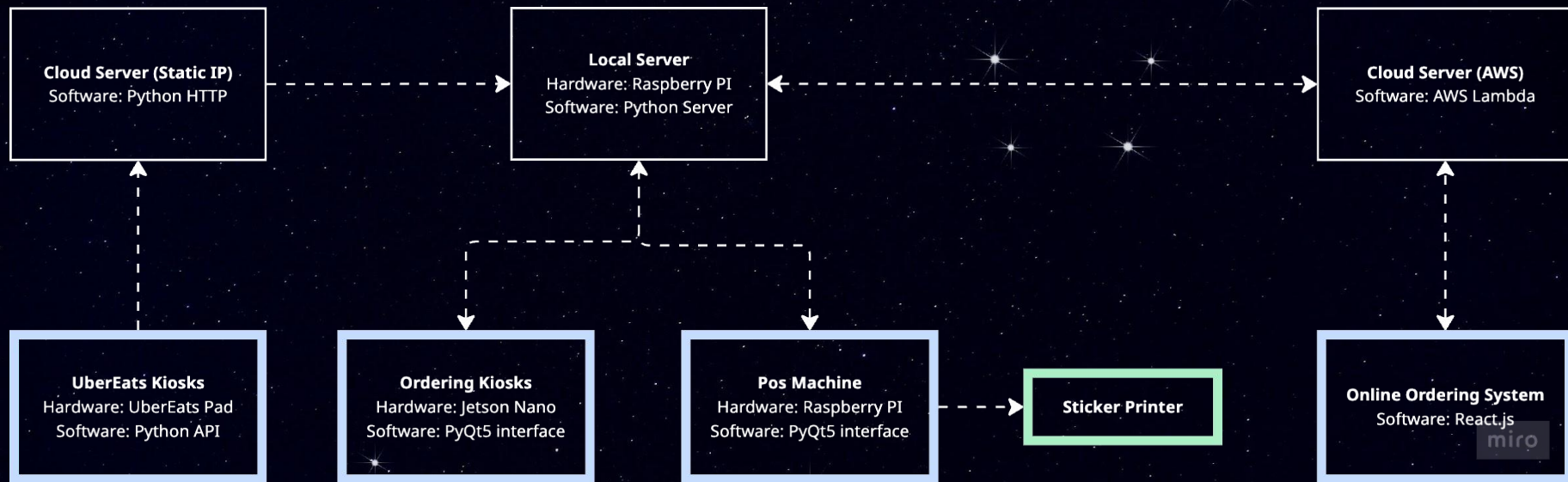
Goal: To build an automated system that reduces the need for staff and digitizes the ordering process.

Our Solution: 4 Order Pathways



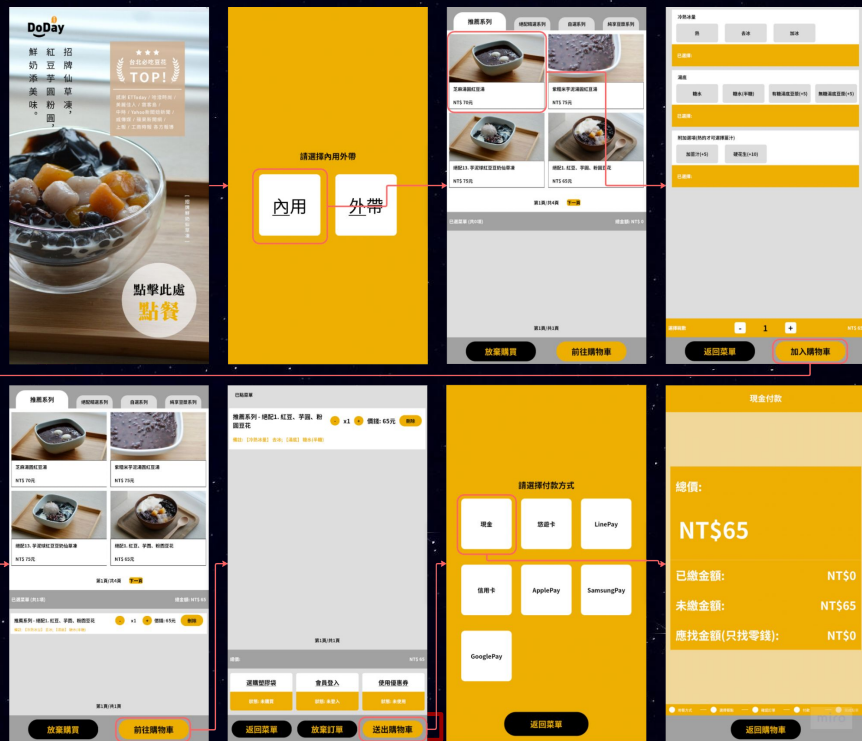
- **Kiosks** (Primary): Automates 90% of walk-in orders.
- **POS** (Fallback): Ensures system redundancy & edge-case handling.
- **UberEats** (Reach): Integrates 3rd-party delivery logistics.
- **Web App** (Margin): Bypasses 30% platform fees for bulk orders.
- **Mission: Route 4 distinct data streams into 1 physical sticker—zero manual entry.**

My Responsibilities



- Led a 5-person engineering team through a 3-year development cycle from concept to deployment.
- Directed the end-to-end system architecture, integrating hardware kiosks, edge servers, and AWS cloud infrastructure.
- Built the physical ordering machines, POS system, and online platform from scratch.
- Designed the hybrid API routing and successfully maintained the live production system for 6 continuous years.

Path 1: Ordering Kiosks

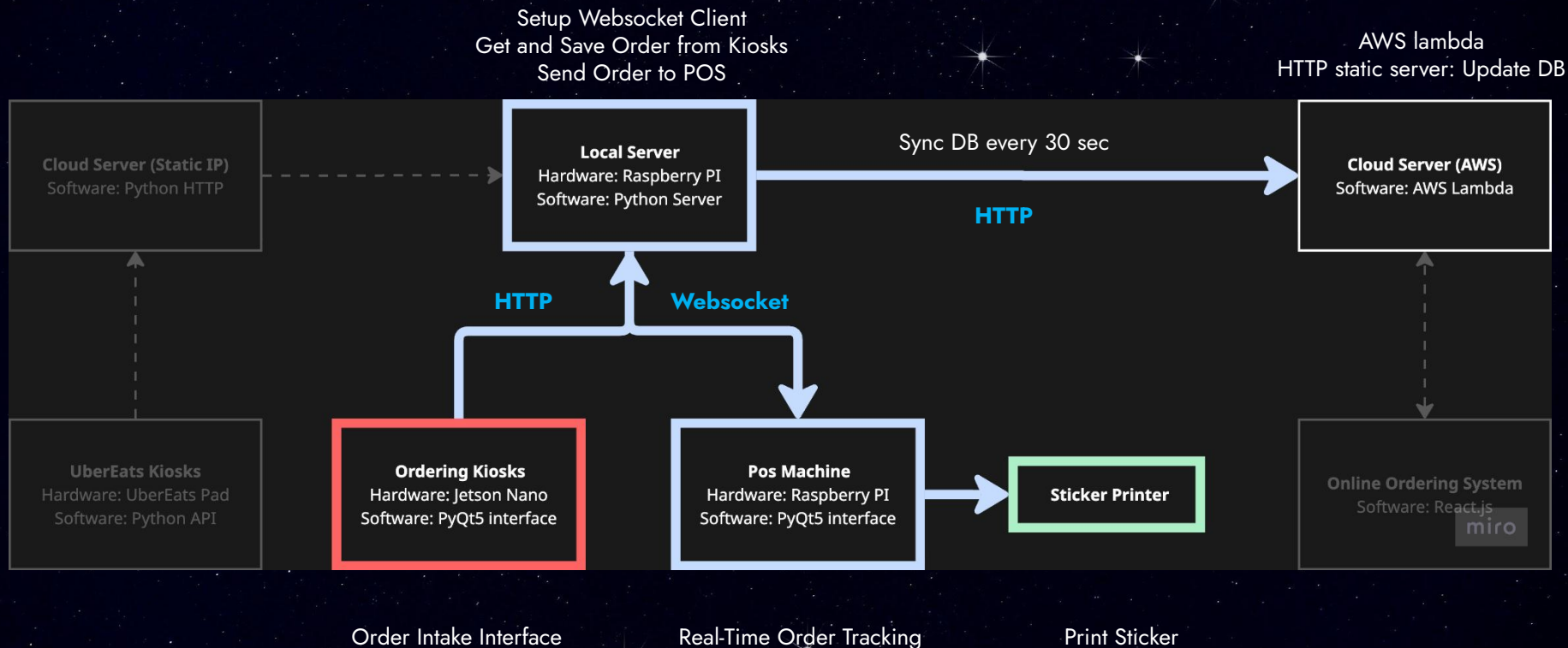


submit_cart_function()

handle_special_day_and_add_to_order_api()



Path 1: Ordering Kiosks



Ordering Kiosks (Code)

proj-005-turkeypro_order_local_machine (doday-devel) - page_handler.py

```
def submit_cart_function(self):
    payment_method =
    self.configs["payment_scheme"][self.page_payment_choice.sender().text(
    )]
```

```
# submit self.submit_data
self.submit_data["payment_method"] = payment_method
```

```
total price = int(self.submit_data["total_price"])
flag, message = self.post_verify_order()
```

Post to Local Server to verify

```
if total_price == 0:
    [# total_price == 0, don't need to jump into PaymentPage]
else:
    if flag: [# if the order is valid]
    else: [# Error order, order again]
```

```
def handle_special_day_and_add_to_order_api(self):
    # split to single items for add to order
    self.submit_data.split_to_single_items()
```

```
[# check if the customer orders any special day]
```

```
if self.submit_data["spin_times"] <= 0:
    # if the customer don't order any special day
    self.post_add_to_order()
```

Post the order to Local Server

```
else:
    # if the customer has ordered special day
    [# find special_day]
```

```
# Call the next routine, start roulette
self.roulette_alert_clock.start()
self.special_day_display_wait_info(self.submit_data["spin_times"])
```

Local Server (Code)

proj-005-cloud-server (doday-stable) - order_helper.py

```
def add_to_order_handler(input_request, **configs):
    # Import order data from json format
    new_order = OrderInfo.json_format_to_order_info(input_request)
```

```
[# Handle order with promotion]
[# Verify business hour and availability]
```

```
# Handle order add for different order types
order_type = new_order.get_order_type()
new_order["machine_id"] = order_type
```

Save Order to Local DB

```
with configs["db_lock"]:
    try:
        # Block the flow by at most 1 second. Avoid race condition
        with time limit(1):
            assign_new_id_and_add_order_to_db(new_order, **configs)
```

```
except TimeoutException as e:
    logging.error("Timed out at assign_order_number and
    add_to_order")
```

```
[# Handle order with promotion]
[# Update different new_order["payment_method"]]
```

Send Order to POS

```
set_status_to_paid_and_send_to_websocket(new_order, **configs)
```

```
return {"indicator": True, "message": "Order successfully added to
the system", "order_id": new_order["order_id"],
        "serial_number": new_order["serial_number"]}
```


POS Machine (Code)

proj-005-kitchen-queue-viewer (doday-stable) - QueuePage.py

```
# Add qt signaling function to ws_client
self.wsclient.add_queue = dict_based_signal_connection()
self.wsclient.add_queue.signal.connect(self.add_queue)

...

def add_queue(self, order_info_dict):
    order_info = OrderInfo.json_format_to_order_info(order_info_dict)
    if order_info in self.queue_data and order_info["order_id"] in
self.queue_button_hashtable:
        return

    order_info_clone = copy.deepcopy(order_info)

    # Handler the case when we need sticker
    sticker = DodayOrderSticker(tmpfile_path=self.sticker_tmpfile_path,
**self.configs)
    deviceprint(self.sticker_printer_name,    Print Sticker
self.sticker_tmpfile_path)(lambda order,**k:
sticker.order_info_to_order_sticker(order))(order_info_clone)
```

```
order_info.merge_same_item()
# Add the order to the queue
self.queue_data.append(order_info)
self.add_order_on_queue_layout(order_info)
```

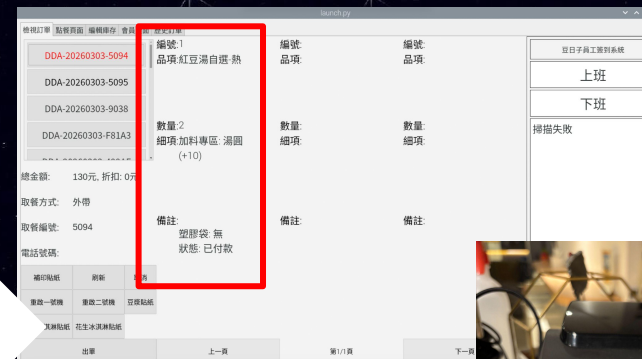
Display the order on POS

```
# Ring the order sound
cli_pi_play_sound(self.ring_sound_path, 'hdmi', mode="normal")
```

```
# Print Invoice
# -----
if self.printer_on:
    self.doday_api.order_print(order_info)
```

This is to print receipt

POS Machine



Path 2: POS Machine

POS Ordering Page

綠豆冰沙豆奶 豆乳霜淇淋

絕配0. 零糖豆漿豆花	絕配1. 紅豆、芋圓、粉圓豆花	熱	去冰	內用	0塊袋
絕配10. 薏仁綠豆湯	絕配11. 綜合黑糖刨冰	微冰	半冰	外帶	1塊袋
絕配12. 招牌鮮奶仙草凍	絕配13. 芋泥球紅豆豆奶仙草凍	少冰	加冰	外送	2塊袋
絕配2. 花生豆花	絕配3. 紅豆、紫米、豆漿豆花	無糖	半糖	貨到付款	3塊袋
絕配4. 滑圓花生紅豆湯	絕配5. 椰奶紅豆紫米粥	花生(+10)	紅豆(+5)	無糖	4塊袋
絕配6. 薏仁紫米粥	絕配7. 椰奶芋圓燒仙草	綠豆(+5)	薏仁(+5)	貨到付數	UberEats
絕配8. 招牌蘭西燒仙草	絕配9. 寒天薏仁粉圓綠豆湯	滿圓(+10)	椰果(+5)	輸入數字: 0	
仙草凍自選	紅豆湯自選-熱	椰奶(+10)	芋圓(+5)	1 2 3	掃描條碼外二條碼:
紫米粥自選	綠豆湯自選-冷	粉圓(+5)	芋頭(+15)	4 5 6	掃描會員條碼, 會員電話:
豆花自選	蘭西燒仙草純底	寒天(+10)	仙草凍(+5)	7 8 9	小計: 0
額外補差價	黑糖刨冰自選	豆花(+10)	芋泥球(+25)	0 del enter	確認訂單 刪除訂單
		粉圓(+10)	綠豆(+5)		送出

綠豆冰沙豆奶 豆乳霜淇淋

DDA-20260303-5094	編號 1	品項 紅豆湯自選-熱	品項	品項
DDA-20260303-5095				
DDA-20260303-9038				
DDA-20260303-F81A3	數量-2	細項 加料專區: 滑圓 (+10)	數量: 細項	數量: 細項

總金額: 130元, 折扣: 0元

取餐方式: 外帶

取餐編號: 5094

電話號碼:

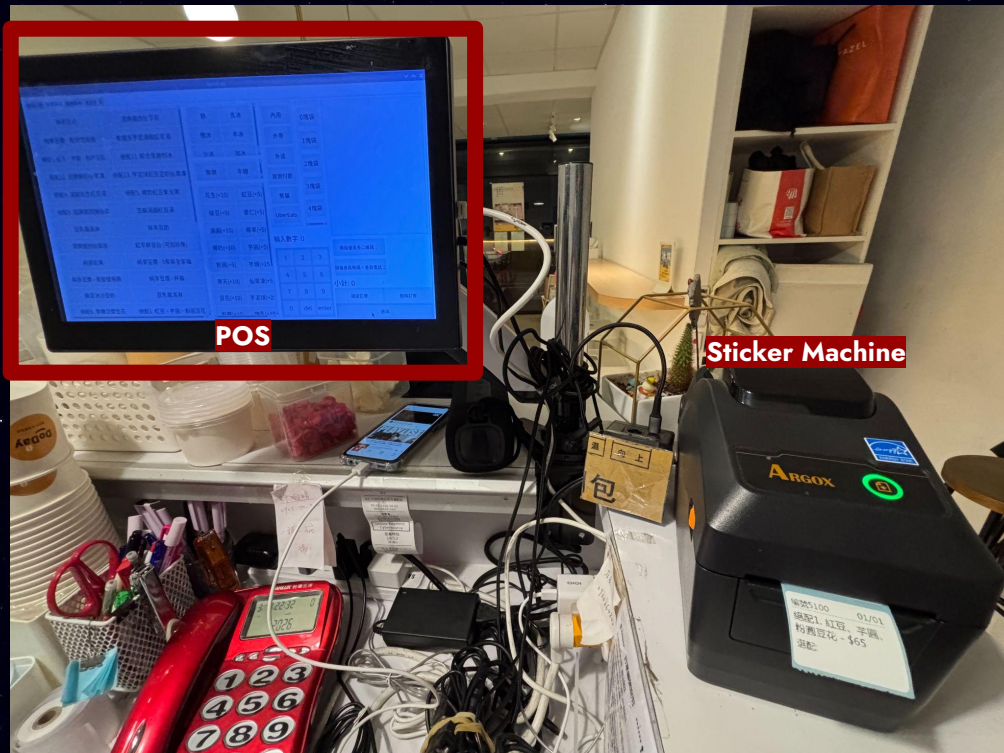
備註: 塑膠袋 無
狀態: 已付款

補印貼紙 斷紙 取消

重啟一號機 重啟二號機 豆漿貼紙

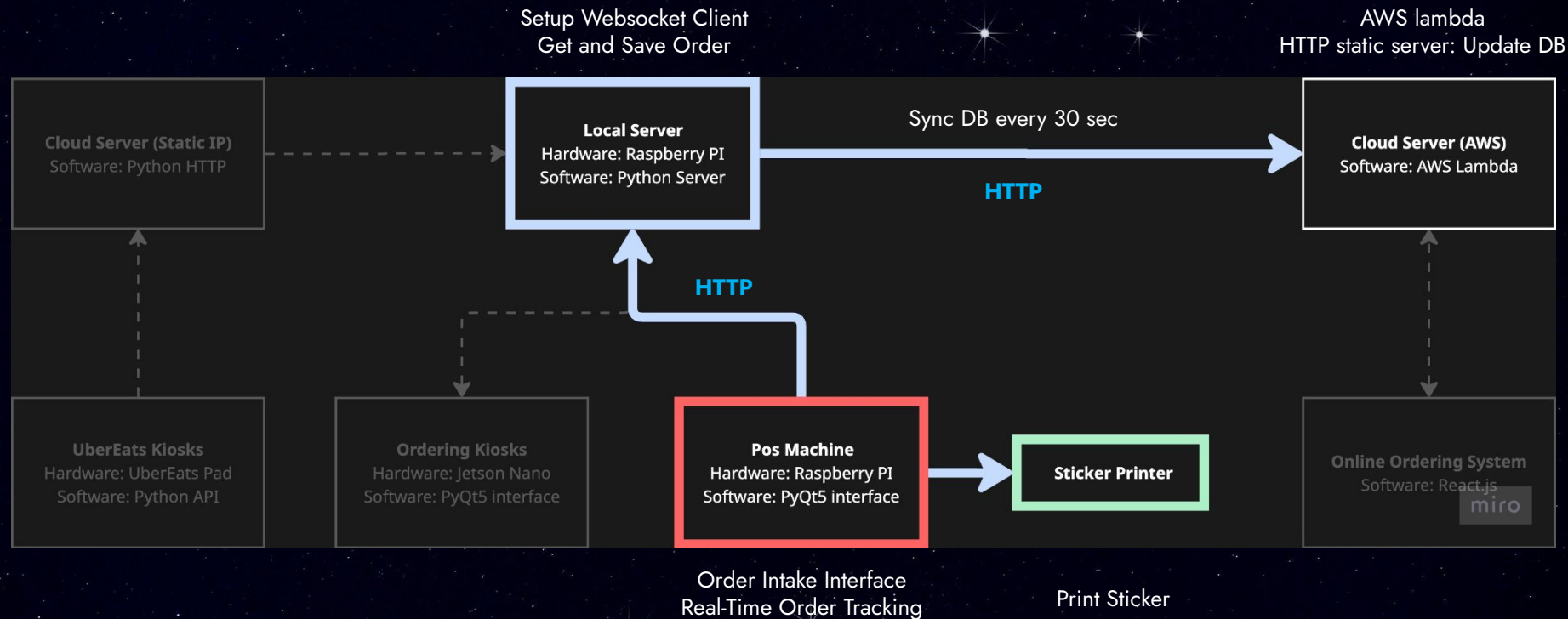
豆漿冰淇淋貼紙 花生冰淇淋貼紙

出單 上一頁 第1頁 下一頁



POS Order Display

Path 2: POS Machine



POS Machine

Screenshot of the POS Machine interface showing the order entry screen. The interface is divided into several sections:

- Top Section:** Includes tabs for "檢視訂單", "點餐頁面", "編輯庫存", "會員頁面", and "歷史訂單". Below these are two columns of product categories: "綠豆冰沙豆奶" and "豆乳霜淇淋".
- Product Selection:** A grid of product options with their respective prices and quantities. For example, "綠豆冰沙豆奶" is priced at 130元, and "豆乳霜淇淋" is priced at 10元.
- Order Entry:** A section for entering the order details, including a "編號" (ID) field, a "品項" (Item) field, and a "數量" (Quantity) field. The "編號" field is highlighted with a red box.
- Payment Section:** Includes fields for "總金額" (Total Amount), "折扣" (Discount), and "支付方式" (Payment Method). The "支付方式" field is set to "外帶" (Takeout).
- Order Summary:** A section on the right side of the screen showing the order details, including the "編號" (ID), "品項" (Item), "數量" (Quantity), and "備註" (Remarks).

Printer



POS Machine (Code)

proj-005-kitchen-queue-viewer (doday-stable) - KitchenOrderPage.py

```
def send_to_server(self, mod, **kwargs):
    if self.edit_order_status == 1:
        # Cannot send order to server if
        # it is in edit mode
        return False
```

```
order_info = self.prepare_order_info()
```

```
post_data = {
    "service": "order",
    "operation": "add_to_order",
}
post_data.update(order_info.tojson())
```

Post the order to Local Server
(Same API with Ordering Kiosks)

```
ret = self.httpclient.client_post("main_server_http_url",
    post_data)
res = json.loads(ret)
```

```
if res["indicator"]:
    [# Reset Order Page UI]
else:
    AlertToast(res["message"]).run()
```

Local Server (Database)

Latest Order

id	order_id	order_no	store_id	serial_number	username	name1	name2	name3	name4	stage	name5	amount	price
1	382819	DDA-20260304-1001	2	DDA	1001	推薦系列	總配13: 芋泥球紅豆豆粉仙草凍	「標準」: 「標準配料」		stay		1	75
2	382818	DDA-20260304-1001	1	DDA	1001	推薦系列	北極海鹽紅豆凍	「標準」: 「標準配料」	附加碼	stay		1	70
3	382817	DDA-20260304-5001	1	DDA	5001	總配推薦系列	總配4: 海鹽花生紅豆凍	「標準」: 「標準配料」	附加碼	stay		1	70
4	382816	DDA-20260304-5000	1	DDA	5000	自選系列	紅豆海鹽自選	「加料專區」: 「字匯」(+/-): 「字匯」		stay		1	65
5	382815	DDA-20260304-7001	23	DDA	7001	總配推薦系列	總配3: 招牌雙西佛仙草	「標準」: 「標準配料」		delivery		1	75
6	382814	DDA-20260304-7001	24	DDA	7001	None	總配8: 招牌雙西佛仙草	「標準」: 「標準配料」		delivery		1	75
7	382813	DDA-20260304-7001	23	DDA	7001	None	總配8: 招牌雙西佛仙草	「標準」: 「標準配料」		delivery		1	75
8	382812	DDA-20260304-7001	22	DDA	7001	None	總配8: 招牌雙西佛仙草	「標準」: 「標準配料」		delivery		1	75
9	382811	DDA-20260304-7001	21	DDA	7001	None	總配8: 招牌雙西佛仙草	「標準」: 「標準配料」		delivery		1	75
10	382810	DDA-20260304-7001	20	DDA	7001	None	總配8: 招牌雙西佛仙草	「標準」: 「標準配料」		delivery		1	75

price	discount	final_price	total_price	payment_method	receipt_number	status	promotion	promotion_key	order_time	print_flag	xml_flag	comment1	comment2	comment3
75	0	75	145	cash		preparing	TEST8		0	0	0	local pay	0910	
75	0	75	70	cash		preparing	TEST8		0	0	0	local pay	0910	
75	0	65	65	cash		preparing	TEST8		0	0	0	local pay	0910	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	

comment3	comment4	comment5	comment6	comment7	comment8	time	sync_flag
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "T0": "0", "D0": "0", "S00": "0")	not updated	('one_time_code': "deducted_point": 0)		1772594626	true

Sync flag

Local Server (Code)

proj-005-cloud-server (doday-stable) - gados_order_sync.py

```
def gados_order_sync(**configs):
    with configs["sync_lock"]:
        # lock the entire sync and update process
        unsynced_dict = get_not_synced_dict()

        ret = json.loads(post_order_sync_to_cloud({"store_id":
configs["store id"], "brand id": configs["brand id"],
        "unsynced_dict": unsynced_dict}, **configs))

        if ret.get("indicator", False):
            update_local_sync_status(unsynced_dict)

            if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("new_orders_from_online", False):

add_order_to_local_server(ret["message"]["new_orders_from_online"],
**configs)

            if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("store_info_from_online", False):

update_store_information_to_local_server(ret["message"]["store_info_fr
om_online"], **configs)

            if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("promotion_info_from_online", False):

update_promotion_to_local_server(ret["message"]["promotion_info_from_o
nline"], **configs)
```

Sync Local to Cloud

AWS Server

PCubeOnlineOrderServer (doday-stable) - order_helper.py

```
def post_order_sync_to_cloud_handler(input_request, **configs):
    store_id, unsynced_dict, brand_id = input_request['store_id'], \
        input_request['unsynced_dict'], input_request["brand_id"]

    [# First check whether the order id is in the database or not]
    data = get_data('order', ['order_id'], where_value_list, multiple=True)
    [# Handle the existing order list and only update the flag]

    existing_order_list_clone = copy.deepcopy(existing_order_list)
    for order_clone in existing_order_list_clone:
        order_clone["sync_flag"] = True

    _ = edit_on_table("order", existing_order_list_clone, ["order_id", "order_no"])

    [# Handle the none existing order and add those to the table]

    _ = add_to_table("order", add_list)

    _ = edit_on_table("order", existing_order_list_clone, ["order_id", "order_no"])

    [# Update promotion key usage and member related info]

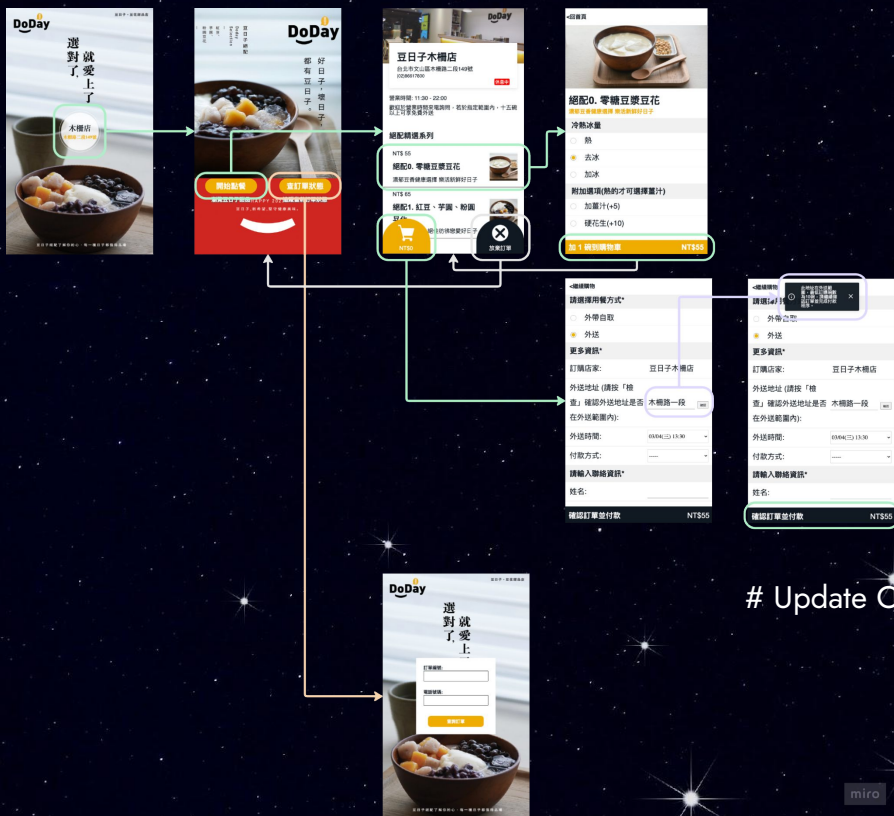
    # Future Modification
    message_dict = {}
    # After syncing the data from local, try to get those from the
    # cloud db that haven't been synced.
    not_synced_cloud_order = get_not_synced_dict(store_id)
    if not_synced_cloud_order == {}:
        # Empty unsynced order
        message_dict.update({"new_orders_from_online": False})
    else:
        message_dict.update({"new_orders_from_online": not_synced_cloud_order})

    return {"indicator": True, "message": message_dict}
```

Save Order to DB

Send not Synced
Order to local

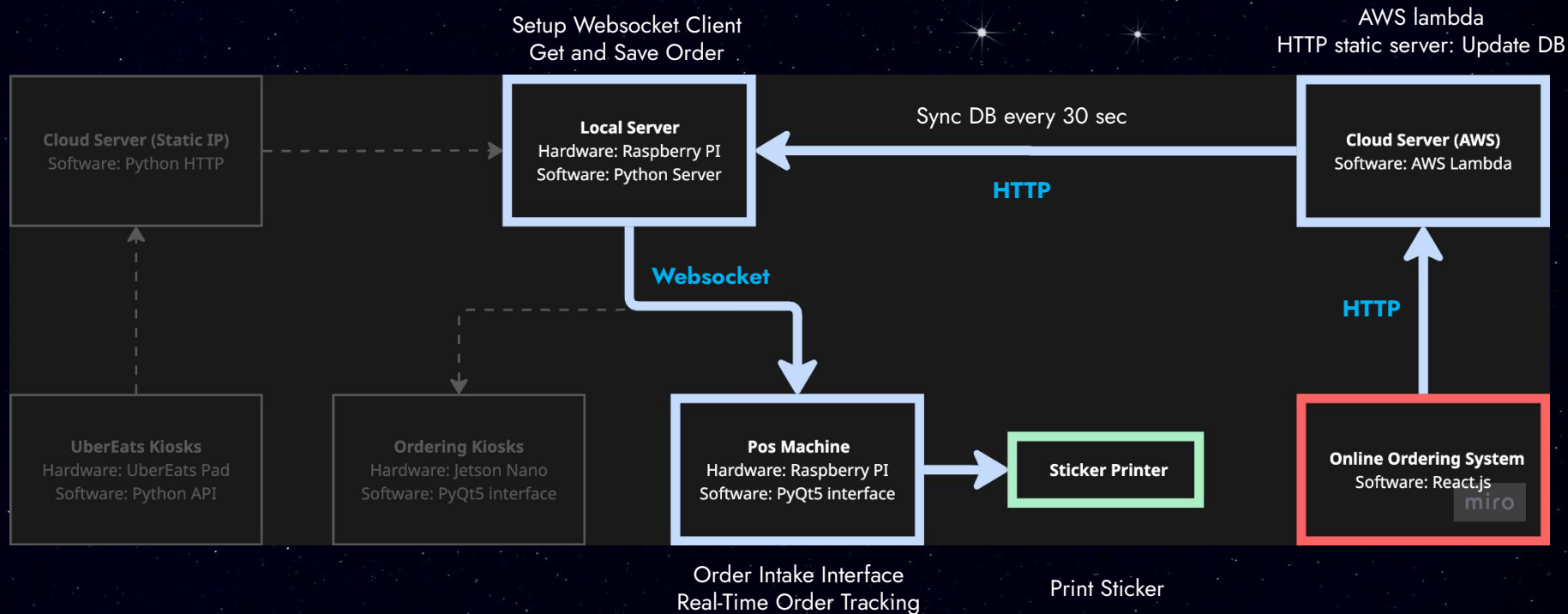
Path 3: Online Ordering System



Update Order to AWS Server



Path 3: Online Ordering System



Online Ordering (Code)

proj-005-react_online_order (doday-devel) - cart_page.js

```

async handleSubmit(e) {
  if (/[// check order])
  else{
    e.target.disabled = true
    [// check member]
    var order_list_data = getCookie("shoppingCartItems");
    var decode_list = await get_cart_item(decode_order_list)
    [// revise format]
    [// Repeatedly add the same data for the same bowl, for number of bowls]

    var request_data = {
      "service": "order",
      "operation": "add to order",
      "name1": category_list,
      "name2": item_list,
      "name3with4": addon_list,
      "name5": comment_list,
      "number_of_data": number_of_bowl.toString(),
      "price": price_list,
      "total_price": this.state.totalPrice.toString(),
      "promotion key": coupon fee,
      "final_price": price_list,
      "payment_method": online or local(this.state.stayOrTogo,
      this.state.extraInfo["payment_method"]),
      "store_id": getCookie("store_id"),
    }

    var request_str = JSON.stringify(request_data)
    postDataFromServer(process.env.REACT_APP_SERVER_URL + '?', request_str)
      .then(result => {[// update web page]})
      .catch(error => {[// switch to error page]})
  }
}

```

Send Order to AWS

AWS Server (Database)

Latest Order

id	order_id	order_no	store_id	serial_number	username	name1	name2	name3with4	stayortogo	name5	amount	price
1	382819	DDA-20260304-1001	2	DDA	1001	推薦系列	總配13 芋泥球紅豆豆沙仙草凍	「標準」[標準配料]	stay		1	75
2	382816	DDA-20260304-1001	1	DDA	1001	推薦系列	芝麻滑溜紅豆湯	「標準」[標準配料] 附加	stay		1	70
3	382817	DDA-20260304-5001	1	DDA	5001	總配精選系列	總配4 滑溜花生紅豆湯	「標準」[標準配料] 附加	togo		1	70
4	382816	DDA-20260304-5000	1	DDA	5000	自選系列	紅豆滑溜自選熱	「加料專區」[芋圓(+5%)、芋圓]	togo		1	65
5	382815	DDA-20260304-7001	25	DDA	7001	總配精選系列	總配3 招牌雙香滑仙草	「標準」[標準配料]	delivery		1	75
6	382814	DDA-20260304-7001	24	DDA	7001	None	總配8 招牌雙香滑仙草	「標準」[標準配料]	delivery		1	75
7	382813	DDA-20260304-7001	23	DDA	7001	None	總配精選系列	總配8 招牌雙香滑仙草	delivery		1	75
8	382812	DDA-20260304-7001	22	DDA	7001	None	總配精選系列	總配8 招牌雙香滑仙草	delivery		1	75
9	382811	DDA-20260304-7001	21	DDA	7001	None	總配精選系列	總配8 招牌雙香滑仙草	delivery		1	75
10	382810	DDA-20260304-7001	20	DDA	7001	None	總配精選系列	總配8 招牌雙香滑仙草	delivery		1	75

price	discount	final_price	total_price	payment_method	receipt_number	status	promotion	promotion_key	order_time	print_flag	xml_flag	comment1	comment2	comment3
75	0	75	145	cash		preparing		TEST8		0		0	local pay	0910
70	0	70	142	cash		preparing		TEST8		0		0	local pay	0910
65	0	65	65	cash		preparing		TEST8		0		0	local pay	0910
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	

comment3	comment4	comment5	comment6	comment7	comment8	time	sync_flag
1	0012711214 ('pick_up_time': 'phone_number': 'delivery_location': ('S0': '1', 'I0': '0', 'S1': '1', 'I1': '0', 'I00': '2', 'I000': '0', 'I0000': '0'))	not updated	['one_time_code': 'deducted_point': 0]			1772596626	true
2	0012711215 ('pick_up_time': 'phone_number': 'delivery_location': ('S0': '1', 'I0': '0', 'S1': '1', 'I1': '0', 'I00': '2', 'I000': '0', 'I0000': '0'))	not updated	['one_time_code': 'deducted_point': 0]			1772596626	true
3	0012711216 ('pick_up_time': 'phone_number': 'delivery_location': ('S0': '0', 'I0': '3', 'S1': '0', 'I1': '0', 'I00': '1', 'I000': '0', 'I0000': '0'))	not updated	['one_time_code': 'deducted_point': 0]			1772596634	true
4	0012711217 ('pick_up_time': 'phone_number': 'delivery_location': ('S0': '1', 'I0': '1', 'S1': '1', 'I1': '0', 'I00': '0', 'I000': '0', 'I0000': '0'))	not updated	['one_time_code': 'deducted_point': 0]			1772597316	true
5	0012711218 ('pick_up_time': '03/04/23 15:00', 'phone_number': '096...')	None	not updated	['one_time_code': 'deducted_point': 0]	None	1772596978	true
6	0012711219 ('pick_up_time': '03/04/23 15:00', 'phone_number': '096...')	None	not updated	['one_time_code': 'deducted_point': 0]	None	1772596978	true
7	0012711220 ('pick_up_time': '03/04/23 15:00', 'phone_number': '096...')	None	not updated	['one_time_code': 'deducted_point': 0]	None	1772596978	true
8	0012711221 ('pick_up_time': '03/04/23 15:00', 'phone_number': '096...')	None	not updated	['one_time_code': 'deducted_point': 0]	None	1772596978	true
9	0012711222 ('pick_up_time': '03/04/23 15:00', 'phone_number': '096...')	None	not updated	['one_time_code': 'deducted_point': 0]	None	1772596978	true
10	0012711223 ('pick_up_time': '03/04/23 15:00', 'phone_number': '096...')	None	not updated	['one_time_code': 'deducted_point': 0]	None	1772596978	true

Sync flag

AWS Server (Code)

PCubeOnlineOrderServer (doday-stable) - order_helper.py

```
def post_order_sync_to_cloud_handler(input_request, **configs):
    store_id, unsynced_dict, brand_id = input_request['store_id'], \
        input_request['unsynced_dict'], input_request['brand_id']

    [# First check whether the order id is in the database or not]
    data = get_data('order', ['order_id'], where_value_list, multiple=True)
    [# Handle the existing order list and only update the flag]

    existing_order_list_clone = copy.deepcopy(existing_order_list)
    for order_clone in existing_order_list_clone:
        order_clone["sync_flag"] = True

    _ = edit_on_table("order", existing_order_list_clone, ["order_id", "order_id"])

    [# Handle the none existing order and add those to the table]

    _ = add_to_table("order", add_list)    Save Order to DB

    [# Update promotion key usage and member related info]

    # Future Modification
    message_dict = {}
    # After syncing the data from local, try to get those from the
    # cloud db that haven't been synced.
    not_synced_cloud_order = get_not_synced_dict(store_id)    Send not Synced
    if not_synced_cloud_order == {}:                            Order to local
        # Empty unsynced order
        message_dict.update({"new_orders_from_online": False})
    else:
        message_dict.update({"new_orders_from_online": not_synced_cloud_order})

    return {"indicator": True, "message": message_dict}
```

Local Server (Code)

proj-005-cloud-server (doday-stable) - gados_order_sync.py

```
def gados_order_sync(**configs):
    with configs["sync_lock"]:
        # lock the entire sync and update process
        unsynced_dict = get_not_synced_dict()
        logging.info("[Gados Order Sync] Syncing: "
            +str(list(unsynced_dict.keys())))
        ret = json.loads(post_order_sync_to_cloud({"store_id":
            configs["store_id"], "brand id": configs["brand id"],
            "unsynced_dict": unsynced_dict}, **configs))

        if ret.get("indicator", False):
            update_local_sync_status(unsynced_dict)

            if isinstance(ret.get("message", False), dict) and
                ret.get("message", {}).get("new_orders_from_online", False):

                add_order_to_local_server(ret["message"]["new_orders_from_online"],
                    **configs)

                if isinstance(ret.get("message", False), dict) and
                    ret.get("message", {}).get("store_info_from_online", False):

                    update_store_information_to_local_server(ret["message"]["store_info_fr
                        om_online"], **configs)

                    if isinstance(ret.get("message", False), dict) and
                        ret.get("message", {}).get("promotion_info_from_online", False):

                        update_promotion_to_local_server(ret["message"]["promotion_info_from_o
                            nline"], **configs)
```

Sync Cloud to Local

Local Server (Code)

proj-005-cloud-server (doday-stable) - gados_order_sync.py

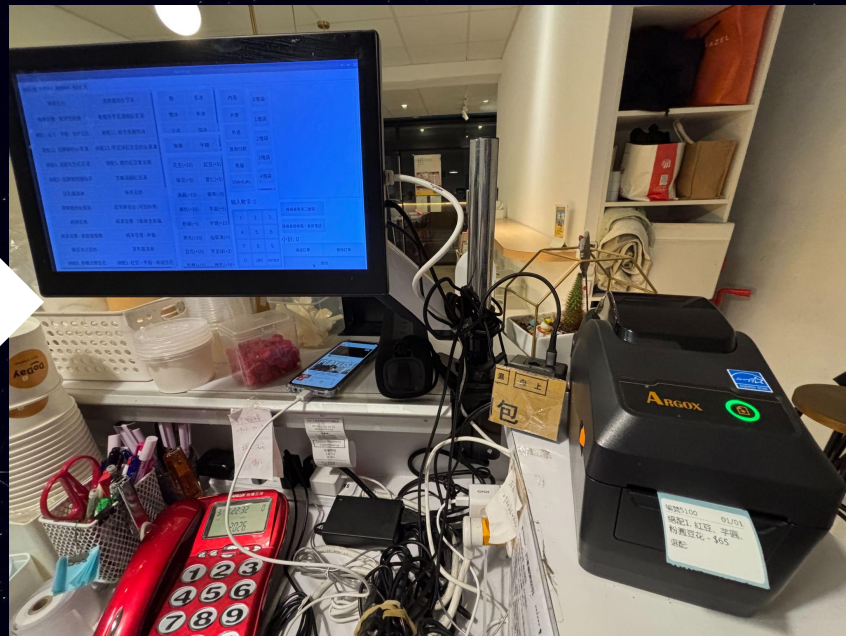
```
def add_order_to_local_server(payload, **configs):
    order_id_list = list(payload.keys())
    where_value_list = []
    for oid in order_id_list:
        where_value_list.append((oid,))

    matched_data = get_data("order", ["order_id"], where_value_list,
                             multiple=True)
    exist_order_id_set = set()
    for data in matched_data:
        exist_order_id_set.add(data["order_id"])

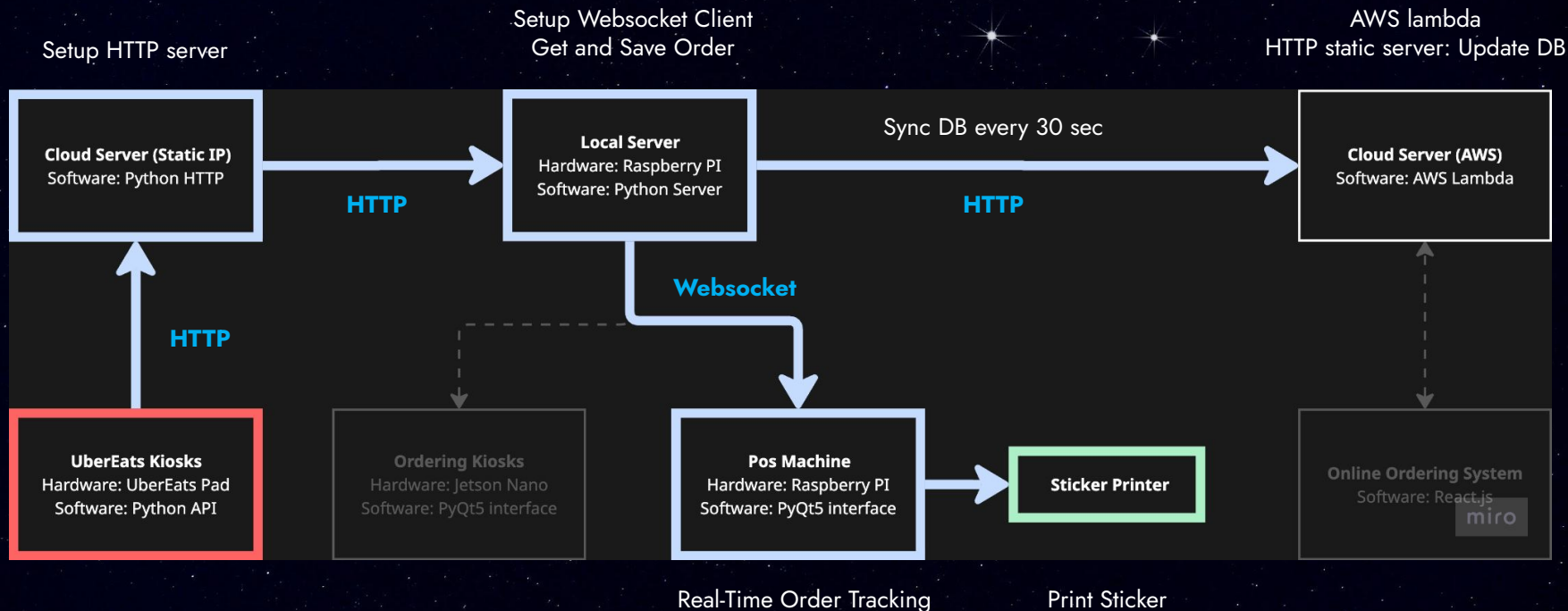
    for order_id in payload:
        if order_id in exist_order_id_set:
            continue
        order_info = OrderInfo.db_format_to_order_info(payload[order_id])
        order_info.sync_flag = True
        dict_data = order_info.tojson()
        dict_data.update({"service": "order", "operation": "add_to_order"})
        client = GadgetHiClient(**configs)
        _ = client.client_post("local_server_http_url", dict_data)
```

Post the order to Local Server
(Same API with Ordering Kiosks)

POS Machine



Path 4: UberEats



Cloud Server (Code)

proj-002-uber_eats_integration (doday-stable) - server.py

```
def main_handler(cls, event, **configs):
    uberEatsManager = UberEats(configs["uber_client_id"],
                                configs["uber_client_secret"], store_id,
                                configs["uber_eats_secret_filepath"])

    [# Verify whether the information comes from Uber]

    [# change the details to orderinfo]
    order_details = uberEatsManager.get_order_details(order_id)
    uber_order_info = uber_eats_order_to_order_info(order_details,
    uber_mapping)

    post_data = {
        "service": "order",
        "operation": "uber_add_to_order",
    }
    post_data.update(uber_order_info.tojson())

    if store_id in uber_store_id_to_doday_store_id_map.keys():
        client = DodayHttpClient(**configs)
        ret = client.client_post("uber http url", post_data, gauth=True,
                                gadgethi_key=configs["gadgethi_key"],gadgethi_secret=configs["gadgethi_secret"])
        res = json.loads(ret)

    return {'statusCode': 200, 'body': 'Successful from Uber Server'}
```

Local Server (Database)

Latest Order

id	order_id	order_no	store_id	serial_number	username	name1	name2	name3/ID4	stage/ptgo	name5	amount	price
1	382819	DDA-20260304-1001	2	DDA	1001	推薦系列	結配13: 芋泥球紅豆亞的仙草凍	「標準」:「標準配料」	stay		1	75
2	382816	DDA-20260304-1001	1	DDA	1001	推薦系列	芝麻滑溜紅豆湯	「標準」:「標準配料」,「附加」	stay		1	70
3	382817	DDA-20260304-5001	1	DDA	5001	結配精選系列	結配4: 滿園花生紅豆湯	「標準」:「標準配料」,「附加」	togo		1	70
4	382816	DDA-20260304-5000	1	DDA	5000	自選系列	紅豆滑溜自選熱	「附加」:「標準」:「字匯」(+/-):「字匯」	togo		1	65
5	382815	DDA-20260304-7001	23	DDA	7001	結配精選系列	結配3: 招牌雙西德仙草	「標準」:「標準配料」	delivery		1	75
6	382814	DDA-20260304-7001	24	DDA	7001	結配精選系列	結配3: 招牌雙西德仙草	「標準」:「標準配料」	delivery		1	75
7	382813	DDA-20260304-7001	23	DDA	7001	結配精選系列	結配3: 招牌雙西德仙草	「標準」:「標準配料」	delivery		1	75
8	382812	DDA-20260304-7001	22	DDA	7001	結配精選系列	結配3: 招牌雙西德仙草	「標準」:「標準配料」	delivery		1	75
9	382811	DDA-20260304-7001	21	DDA	7001	結配精選系列	結配3: 招牌雙西德仙草	「標準」:「標準配料」	delivery		1	75
10	382810	DDA-20260304-7001	20	DDA	7001	結配精選系列	結配3: 招牌雙西德仙草	「標準」:「標準配料」	delivery		1	75

price	discount	final_price	total_price	payment_method	receipt_number	status	promotion	promotion_key	order_time	print_flag	xml_flag	comment1	comment2	comment3
75	0	75	145	cash		preparing	TEST8		0	0	0	local pay	0910	
75	0	75	145	cash		preparing	TEST8		0	0	0	local pay	0910	
75	0	75	145	cash		preparing	TEST8		0	0	0	local pay	0910	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	

comment3	comment4	comment5	comment6	comment7	comment8	time	sync_flag
0912712124	('pick_up_time': "phone_number": "delivery_location":	('S0': "1", "I0": "0", "S1": "1", "I1": "0", "S2": "0", "I2": "0", "S3": "0", "I3": "0", "S4": "0", "I4": "0", "S5": "0", "I5": "0", "S6": "0", "I6": "0", "S7": "0", "I7": "0", "S8": "0", "I8": "0", "S9": "0", "I9": "0", "S10": "0", "I10": "0", "S11": "0", "I11": "0", "S12": "0", "I12": "0", "S13": "0", "I13": "0", "S14": "0", "I14": "0", "S15": "0", "I15": "0", "S16": "0", "I16": "0", "S17": "0", "I17": "0", "S18": "0", "I18": "0", "S19": "0", "I19": "0", "S20": "0", "I20": "0", "S21": "0", "I21": "0", "S22": "0", "I22": "0", "S23": "0", "I23": "0", "S24": "0", "I24": "0", "S25": "0", "I25": "0", "S26": "0", "I26": "0", "S27": "0", "I27": "0", "S28": "0", "I28": "0", "S29": "0", "I29": "0", "S30": "0", "I30": "0", "S31": "0", "I31": "0", "S32": "0", "I32": "0", "S33": "0", "I33": "0", "S34": "0", "I34": "0", "S35": "0", "I35": "0", "S36": "0", "I36": "0", "S37": "0", "I37": "0", "S38": "0", "I38": "0", "S39": "0", "I39": "0", "S40": "0", "I40": "0", "S41": "0", "I41": "0", "S42": "0", "I42": "0", "S43": "0", "I43": "0", "S44": "0", "I44": "0", "S45": "0", "I45": "0", "S46": "0", "I46": "0", "S47": "0", "I47": "0", "S48": "0", "I48": "0", "S49": "0", "I49": "0", "S50": "0", "I50": "0", "S51": "0", "I51": "0", "S52": "0", "I52": "0", "S53": "0", "I53": "0", "S54": "0", "I54": "0", "S55": "0", "I55": "0", "S56": "0", "I56": "0", "S57": "0", "I57": "0", "S58": "0", "I58": "0", "S59": "0", "I59": "0", "S60": "0", "I60": "0", "S61": "0", "I61": "0", "S62": "0", "I62": "0", "S63": "0", "I63": "0", "S64": "0", "I64": "0", "S65": "0", "I65": "0", "S66": "0", "I66": "0", "S67": "0", "I67": "0", "S68": "0", "I68": "0", "S69": "0", "I69": "0", "S70": "0", "I70": "0", "S71": "0", "I71": "0", "S72": "0", "I72": "0", "S73": "0", "I73": "0", "S74": "0", "I74": "0", "S75": "0", "I75": "0", "S76": "0", "I76": "0", "S77": "0", "I77": "0", "S78": "0", "I78": "0", "S79": "0", "I79": "0", "S80": "0", "I80": "0", "S81": "0", "I81": "0", "S82": "0", "I82": "0", "S83": "0", "I83": "0", "S84": "0", "I84": "0", "S85": "0", "I85": "0", "S86": "0", "I86": "0", "S87": "0", "I87": "0", "S88": "0", "I88": "0", "S89": "0", "I89": "0", "S90": "0", "I90": "0", "S91": "0", "I91": "0", "S92": "0", "I92": "0", "S93": "0", "I93": "0", "S94": "0", "I94": "0", "S95": "0", "I95": "0", "S96": "0", "I96": "0", "S97": "0", "I97": "0", "S98": "0", "I98": "0", "S99": "0", "I99": "0", "S100": "0", "I100": "0", "S101": "0", "I101": "0", "S102": "0", "I102": "0", "S103": "0", "I103": "0", "S104": "0", "I104": "0", "S105": "0", "I105": "0", "S106": "0", "I106": "0", "S107": "0", "I107": "0", "S108": "0", "I108": "0", "S109": "0", "I109": "0", "S110": "0", "I110": "0", "S111": "0", "I111": "0", "S112": "0", "I112": "0", "S113": "0", "I113": "0", "S114": "0", "I114": "0", "S115": "0", "I115": "0", "S116": "0", "I116": "0", "S117": "0", "I117": "0", "S118": "0", "I118": "0", "S119": "0", "I119": "0", "S120": "0", "I120": "0", "S121": "0", "I121": "0", "S122": "0", "I122": "0", "S123": "0", "I123": "0", "S124": "0", "I124": "0", "S125": "0", "I125": "0", "S126": "0", "I126": "0", "S127": "0", "I127": "0", "S128": "0", "I128": "0", "S129": "0", "I129": "0", "S130": "0", "I130": "0", "S131": "0", "I131": "0", "S132": "0", "I132": "0", "S133": "0", "I133": "0", "S134": "0", "I134": "0", "S135": "0", "I135": "0", "S136": "0", "I136": "0", "S137": "0", "I137": "0", "S138": "0", "I138": "0", "S139": "0", "I139": "0", "S140": "0", "I140": "0", "S141": "0", "I141": "0", "S142": "0", "I142": "0", "S143": "0", "I143": "0", "S144": "0", "I144": "0", "S145": "0", "I145": "0", "S146": "0", "I146": "0", "S147": "0", "I147": "0", "S148": "0", "I148": "0", "S149": "0", "I149": "0", "S150": "0", "I150": "0", "S151": "0", "I151": "0", "S152": "0", "I152": "0", "S153": "0", "I153": "0", "S154": "0", "I154": "0", "S155": "0", "I155": "0", "S156": "0", "I156": "0", "S157": "0", "I157": "0", "S158": "0", "I158": "0", "S159": "0", "I159": "0", "S160": "0", "I160": "0", "S161": "0", "I161": "0", "S162": "0", "I162": "0", "S163": "0", "I163": "0", "S164": "0", "I164": "0", "S165": "0", "I165": "0", "S166": "0", "I166": "0", "S167": "0", "I167": "0", "S168": "0", "I168": "0", "S169": "0", "I169": "0", "S170": "0", "I170": "0", "S171": "0", "I171": "0", "S172": "0", "I172": "0", "S173": "0", "I173": "0", "S174": "0", "I174": "0", "S175": "0", "I175": "0", "S176": "0", "I176": "0", "S177": "0", "I177": "0", "S178": "0", "I178": "0", "S179": "0", "I179": "0", "S180": "0", "I180": "0", "S181": "0", "I181": "0", "S182": "0", "I182": "0", "S183": "0", "I183": "0", "S184": "0", "I184": "0", "S185": "0", "I185": "0", "S186": "0", "I186": "0", "S187": "0", "I187": "0", "S188": "0", "I188": "0", "S189": "0", "I189": "0", "S190": "0", "I190": "0", "S191": "0", "I191": "0", "S192": "0", "I192": "0", "S193": "0", "I193": "0", "S194": "0", "I194": "0", "S195": "0", "I195": "0", "S196": "0", "I196": "0", "S197": "0", "I197": "0", "S198": "0", "I198": "0", "S199": "0", "I199": "0", "S200": "0", "I200": "0", "S201": "0", "I201": "0", "S202": "0", "I202": "0", "S203": "0", "I203": "0", "S204": "0", "I204": "0", "S205": "0", "I205": "0", "S206": "0", "I206": "0", "S207": "0", "I207": "0", "S208": "0", "I208": "0", "S209": "0", "I209": "0", "S210": "0", "I210": "0", "S211": "0", "I211": "0", "S212": "0", "I212": "0", "S213": "0", "I213": "0", "S214": "0", "I214": "0", "S215": "0", "I215": "0", "S216": "0", "I216": "0", "S217": "0", "I217": "0", "S218": "0", "I218": "0", "S219": "0", "I219": "0", "S220": "0", "I220": "0", "S221": "0", "I221": "0", "S222": "0", "I222": "0", "S223": "0", "I223": "0", "S224": "0", "I224": "0", "S225": "0", "I225": "0", "S226": "0", "I226": "0", "S227": "0", "I227": "0", "S228": "0", "I228": "0", "S229": "0", "I229": "0", "S230": "0", "I230": "0", "S231": "0", "I231": "0", "S232": "0", "I232": "0", "S233": "0", "I233": "0", "S234": "0", "I234": "0", "S235": "0", "I235": "0", "S236": "0", "I236": "0", "S237": "0", "I237": "0", "S238": "0", "I238": "0", "S239": "0", "I239": "0", "S240": "0", "I240": "0", "S241": "0", "I241": "0", "S242": "0", "I242": "0", "S243": "0", "I243": "0", "S244": "0", "I244": "0", "S245": "0", "I245": "0", "S246": "0", "I246": "0", "S247": "0", "I247": "0", "S248": "0", "I248": "0", "S249": "0", "I249": "0", "S250": "0", "I250": "0", "S251": "0", "I251": "0", "S252": "0", "I252": "0", "S253": "0", "I253": "0", "S254": "0", "I254": "0", "S255": "0", "I255": "0", "S256": "0", "I256": "0", "S257": "0", "I257": "0", "S258": "0", "I258": "0", "S259": "0", "I259": "0", "S260": "0", "I260": "0", "S261": "0", "I261": "0", "S262": "0", "I262": "0", "S263": "0", "I263": "0", "S264": "0", "I264": "0", "S265": "0", "I265": "0", "S266": "0", "I266": "0", "S267": "0", "I267": "0", "S268": "0", "I268": "0", "S269": "0", "I269": "0", "S270": "0", "I270": "0", "S271": "0", "I271": "0", "S272": "0", "I272": "0", "S273": "0", "I273": "0", "S274": "0", "I274": "0", "S275": "0", "I275": "0", "S276": "0", "I276": "0", "S277": "0", "I277": "0", "S278": "0", "I278": "0", "S279": "0", "I279": "0", "S280": "0", "I280": "0", "S281": "0", "I281": "0", "S282": "0", "I282": "0", "S283": "0", "I283": "0", "S284": "0", "I284": "0", "S285": "0", "I285": "0", "S286": "0", "I286": "0", "S287": "0", "I287": "0", "S288": "0", "I288": "0", "S289": "0", "I289": "0", "S290": "0", "I290": "0", "S291": "0", "I291": "0", "S292": "0", "I292": "0", "S293": "0", "I293": "0", "S294": "0", "I294": "0", "S295": "0", "I295": "0", "S296": "0", "I296": "0", "S297": "0", "I297": "0", "S298": "0", "I298": "0", "S299": "0", "I299": "0", "S300": "0", "I300": "0", "S301": "0", "I301": "0", "S302": "0", "I302": "0", "S303": "0", "I303": "0", "S304": "0", "I304": "0", "S305": "0", "I305": "0", "S306": "0", "I306": "0", "S307": "0", "I307": "0", "S308": "0", "I308": "0", "S309": "0", "I309": "0", "S310": "0", "I310": "0", "S311": "0", "I311": "0", "S312": "0", "I312": "0", "S313": "0", "I313": "0", "S314": "0", "I314": "0", "S315": "0", "I315": "0", "S316": "0", "I316": "0", "S317": "0", "I317": "0", "S318": "0", "I318": "0", "S319": "0", "I319": "0", "S320": "0", "I320": "0", "S321": "0", "I321": "0", "S322": "0", "I322": "0", "S323": "0", "I323": "0", "S324": "0", "I324": "0", "S325": "0", "I325": "0", "S326": "0", "I326": "0", "S327": "0", "I327": "0", "S328": "0", "I328": "0", "S329": "0", "I329": "0", "S330": "0", "I330": "0", "S331": "0", "I331": "0", "S332": "0", "I332": "0", "S333": "0", "I333": "0", "S334": "0", "I334": "0", "S335": "0", "I335": "0", "S336": "0", "I336": "0", "S337": "0", "I337": "0", "S338": "0", "I338": "0", "S339": "0", "I339": "0", "S340": "0", "I340": "0", "S341": "0", "I341": "0", "S342": "0", "I342": "0", "S343": "0", "I343": "0", "S344": "0", "I344": "0", "S345": "0", "I345": "0", "S346": "0", "I346": "0", "S347": "0", "I347": "0", "S348": "0", "I348": "0", "S349": "0", "I349": "0", "S350": "0", "I350": "0", "S351": "0", "I351": "0", "S352": "0", "I352": "0", "S353": "0", "I353": "0", "S354": "0", "I354": "0", "S355": "0", "I355": "0", "S356": "0", "I356": "0", "S357": "0", "I357": "0", "S358": "0", "I358": "0", "S359": "0", "I359": "0", "S360": "0", "I360": "0", "S361": "0", "I361": "0", "S362": "0", "I362": "0", "S363": "0", "I363": "0", "S364": "0", "I364": "0", "S365": "0", "I365": "0", "S366": "0", "I366": "0", "S367": "0", "I367": "0", "S368": "0", "I368": "0", "S369": "0", "I369": "0", "S370": "0", "I370": "0", "S371": "0", "I371": "0", "S372": "0", "I372": "0", "S373": "0", "I373": "0", "S374": "0", "I374": "0", "S375": "0", "I375": "0", "S376": "0", "I376": "0", "S377": "0", "I377": "0", "S378": "0", "I378": "0", "S379": "0", "I379": "0", "S380": "0", "I380": "0", "S381": "0", "I381": "0", "S382": "0", "I382": "0", "S383": "0", "I383": "0", "S384": "0", "I384": "0", "S385": "0", "I385": "0", "S386": "0", "I386": "0", "S387": "0", "I387": "0", "S388": "0", "I388": "0", "S389": "0", "I389": "0", "S390": "0", "I390": "0", "S391": "0", "I391": "0", "S392": "0", "I392": "0", "S393": "0", "I393": "0", "S394": "0", "I394": "0", "S395": "0", "I395": "0", "S396": "0", "I396": "0", "S397": "0", "I397": "0", "S398": "0", "I398": "0", "S399": "0", "I399": "0", "S400": "0", "I400": "0", "S401": "0", "I401": "0", "S402": "0", "I402": "0", "S403": "0", "I403": "0", "S404": "0", "I404": "0", "S405": "0", "I405": "0", "S406": "0", "I406": "0", "S407": "0", "I407": "0", "S408": "0", "I408": "0", "S409": "0", "I409": "0", "S410": "0", "I410": "0", "S411": "0", "I411": "0", "S412": "0", "I412": "0", "S413": "0", "I413": "0", "S414": "0", "I414": "0", "S415": "0", "I415": "0", "S416": "0", "I416": "0", "S417": "0", "I417": "0", "S418": "0", "I418": "0", "S419": "0", "I419": "0", "S420": "0", "I420": "0", "S421": "0", "I421": "0", "S422": "0", "I422": "0", "S423": "0", "I423": "0", "S424": "0", "I424": "0", "S425": "0", "I425": "0", "S426": "0", "I426": "0", "S427": "0", "I427": "0", "S428": "0", "I428": "0", "S429": "0", "I429": "0", "S430": "0", "I430": "0", "S431": "0", "I431": "0", "S432": "0", "I432": "0", "S433": "0", "I433": "0", "S434": "0", "I434": "0", "S435": "0", "I435": "0", "S436": "0", "I436": "0", "S437": "0", "I437": "0", "S438": "0", "I438": "0", "S439": "0", "I439": "0", "S440": "0", "I440": "0", "S441": "0", "I441": "0", "S442": "0", "I442": "0", "S443": "0", "I443": "0", "S444": "0", "I444": "0", "S445": "0", "I445": "0", "S446": "0", "I446": "0", "S447": "0", "I447": "0", "S448": "0", "I448": "0", "S449": "0", "I449": "0", "S450": "0", "I450": "0", "S451": "0", "I451": "0", "S452": "0", "I452": "0", "S453": "0", "I453": "0", "S454": "0", "I454": "0", "S455": "0", "I455": "0", "S456": "0", "I456": "0", "S457": "0", "I457": "0", "S458": "0", "I458": "0", "S459": "0", "I459": "0", "S460": "0", "I460": "0", "S461": "0", "I461": "0", "S462": "0", "I462": "0", "S463": "0", "I463": "0", "S464": "0", "I464": "0", "S465": "0", "I465": "0", "S466": "0", "I466": "0", "S467": "0", "I467": "0", "S468": "0", "I468": "0", "S469": "0", "I469": "0", "S470": "0", "I470": "0", "S471": "0", "I471": "0", "S472": "0", "I472": "0", "S473": "0", "I473": "0", "S474": "0", "I474": "0", "S475": "0", "I475": "0", "S476": "0", "I476": "0", "S477": "0", "I477": "0", "S478": "0", "I478": "0", "S479": "0", "I479": "0", "S480": "0", "I480": "0", "S481": "0", "I481": "0", "S482": "0", "I482": "0", "S483": "0", "I483": "0", "S484": "0", "I484": "0", "S485": "0", "I485": "0", "S486": "0", "I486": "0", "S487": "0", "I487": "0", "S488": "0", "I488": "0", "S489": "0", "I489": "0", "S490": "0", "I490": "0", "S491": "0", "I491": "0", "S492": "0", "I492": "0", "S493": "0", "I493": "0", "S494": "0", "I494": "0", "S495": "0", "I495": "0", "S496": "0", "I496": "0", "S497": "0", "I497": "0", "S498": "0", "I498": "0", "S499": "0", "I499": "0", "S500": "0", "I500": "0", "S501": "0", "I501": "0", "S502": "0", "I502": "0", "S503": "0", "I503": "0", "S504": "0", "I504": "0", "S505": "0", "I505": "0", "S506": "0", "I506": "0", "S507": "0", "I507": "0", "S508": "0", "I508": "0", "S509": "0", "I509": "0", "S510": "0", "I510": "0", "S511": "0", "I511": "0", "S512": "0", "I512": "0", "S513": "0", "I513": "0", "S514": "0", "I514": "0", "S515": "0", "I515": "0", "S516": "0", "I516": "0", "S517": "0", "I517": "0", "S518": "0", "I518": "0", "S519": "0", "I519": "0", "S520": "0", "I520": "0", "S521": "0", "I521": "0", "S522": "0", "I522": "0", "S523": "0", "I523": "0", "S524": "0", "I524": "0", "S525": "0", "I525": "0", "S526": "0", "I526": "0", "S527": "0", "I527": "0", "S528": "0", "I528": "0", "S529": "0", "I529": "0", "S530": "0", "I530": "0", "S531": "0", "I531": "0", "S532": "0", "I532": "0", "S533": "0", "I533": "0", "S534": "0", "I534": "0", "S535": "0", "I535": "0", "S536": "0", "I536": "0", "S537": "0", "I537": "0", "S538": "0", "I538": "0", "S539": "0", "I539": "0", "S540": "0", "I540": "0", "S541": "0", "I541": "0", "S542": "0", "I542": "0", "S543": "0", "I543": "0", "S544": "0", "I544": "0", "S545": "0", "I545": "0", "S546": "0", "I546": "0", "S547": "0", "I547": "0", "S548": "0", "I548": "0", "S549": "0", "I549": "0", "S550": "0", "I550": "0", "S551": "0", "I551": "0", "S552": "0", "I552": "0", "S553": "0", "I553": "0", "S554": "0", "I554": "0", "S555": "0", "I555": "0", "S556": "0", "I556": "0", "S557": "0", "I557": "0", "S558": "0", "I558": "0", "S559": "0", "I559": "0", "S560": "0", "I560": "0", "S561": "0", "I561": "0", "S562": "0", "I562": "0", "S563": "0", "I563": "0", "S564": "0", "I564": "0", "S565": "0", "I565": "0", "S566": "0", "I566": "0", "S567": "0", "I567": "0", "S568": "0", "I568": "0", "S569": "0", "I569": "0", "S570": "0", "I570": "0", "S571": "0", "I571": "0", "S572": "0", "I572": "0", "S573": "0", "I573": "0", "S574": "0", "I574": "0", "S575": "0", "I575": "0", "S576": "0", "I576": "0", "S577": "0", "I577": "0", "S578": "0", "I578": "0", "S579": "0", "I579": "0", "S580": "0", "I580": "0", "S581": "0", "I581": "0", "S582": "0", "I582": "0", "S583": "0", "I583": "0", "S584": "0", "I584": "0", "S585": "0", "I585": "0", "S586": "0", "I586": "0", "S587": "0", "I587": "0", "S588": "0", "I588": "0", "S589": "0", "I589": "0", "S590": "0", "I590": "0", "S591": "0", "I591": "0", "S592": "0", "I592": "0", "S593": "0", "I593": "0", "S594": "0", "I594": "0", "S595": "0", "I595": "0", "S596": "0", "I596": "0", "S597": "0", "I597": "0", "S598": "0", "I598": "0", "S599": "0", "I599": "0", "S600": "0", "I600": "0", "S601": "0", "I601": "0", "S602": "0", "I602": "0", "S603": "0", "I603": "0", "S604": "0", "I604": "0", "S605": "0", "I605": "0", "S606": "0", "I606": "0", "S607": "0", "I607": "0", "S608": "0", "I608": "0", "S609": "0", "I609": "0", "S610": "0", "I610": "0", "S611": "0", "I611": "0", "S612": "0", "I612": "0", "S613": "0", "I613": "0", "S614": "0", "I614": "0", "S615": "0", "I615": "0", "S616": "0", "I616": "0", "S617": "0", "I617": "0", "S618": "0", "I618": "0", "S619": "0", "I619": "0", "S620": "0", "I620": "0", "S621": "0", "I621": "0", "S622": "0", "I622": "0", "S623": "0", "I623": "0", "S624": "0", "I624": "0", "S625": "0", "I625": "0", "S626": "0", "I626": "0", "S627": "0", "I627": "0", "S628": "0", "I628": "0", "S629": "0", "I629": "0", "S630": "0", "I630": "0", "S631": "0", "I631": "0", "S632": "0", "I632": "0", "S633": "0", "I633": "0", "S634": "0", "I634": "0", "S635": "0", "I635": "0", "S636": "0", "I636": "0", "S637": "0", "I637": "0", "S638": "0", "I638": "0", "S639": "0", "I639": "0", "S640": "0", "I640": "0", "S641": "0", "I641": "0", "S642": "0", "I642": "0", "S643": "0", "I643": "0", "S644": "0", "I644": "0", "S645": "0", "I645": "0", "S646": "0", "I646": "0", "S647": "0", "I647": "0", "S648": "0", "I648": "0", "S649": "0", "I649": "0", "S650": "0", "I650": "0", "S651": "0", "I651": "0", "S652": "0", "I652": "0", "S653": "0", "I653": "0", "S654": "0", "I654": "0", "S655": "0", "I655": "0", "S656": "0", "I656": "0", "S657": "0", "I657": "0", "S658": "0", "I658": "0", "S659": "0", "I659": "0", "S660": "0", "I660": "0", "S661": "0", "I661": "0", "S662": "0", "I662": "0", "S663": "0", "I663": "0", "S664": "0", "I664": "0", "S665": "0", "I665": "0", "S666": "0", "I666": "0", "S667": "0", "I667": "0", "S668": "0", "I668": "0", "S669": "0", "I669": "0", "S670": "0", "I670": "0", "S671": "0", "I671": "0", "S672": "0", "I672": "0", "S673": "0", "I673": "0", "S674": "0", "I674": "0", "S675": "0", "I675": "0", "S676": "0", "I676": "0", "S677": "0", "I677": "0", "S678": "0", "I678": "0", "S679": "0", "I679": "0", "S680": "0", "I680": "0", "S681": "0", "I681": "0", "S682": "0", "I682": "0", "S683": "0", "I683": "0", "S684": "0", "I684": "0", "S685": "0", "I685": "0", "S686": "0", "I686": "0", "S687": "0", "I687": "0", "S688": "0", "I688": "0", "S689": "0", "I689": "0", "S690": "0", "I690": "0", "S691": "0", "I691": "0", "S692": "0", "I692": "0", "S693": "0", "I693": "0", "S694": "0", "I694": "					

Local Server (Code)

proj-005-cloud-server (doday-stable) - server.py

```
def uber_add_to_order_handler(input_request, **configs):
    new_order = OrderInfo.json_format_to_order_info(input_request)

    [# Verify availability]

    # Check if it satisfies uber order convention
    assert "UBER-" in new_order["order_id"]
    uber_display_id = new_order["order_id"].replace("UBER-", "")

    today_date = datetime.date.today().strftime("%Y%m%d")

    assigned_order_id =
    configs["store_id"]+'-'+today_date+'-'+uber_display_id

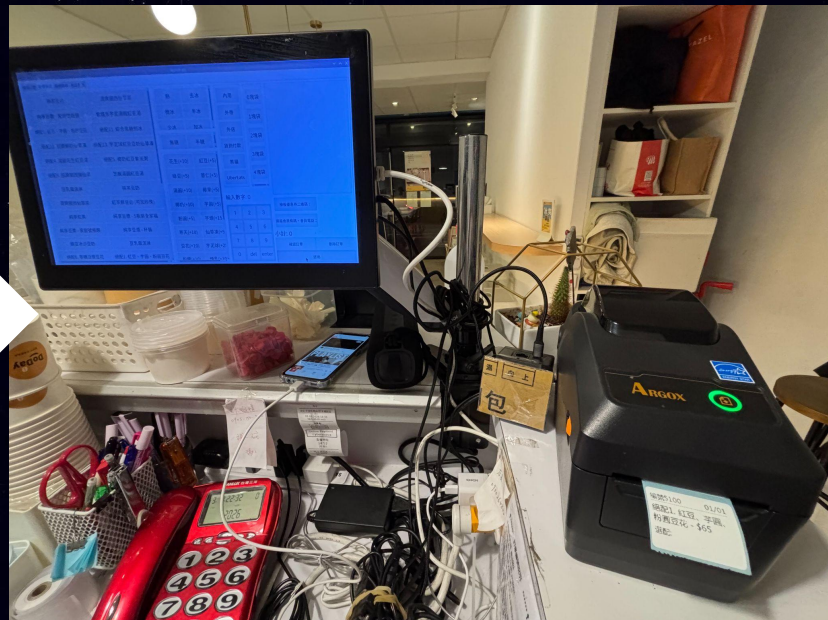
    # assign id and no to order info
    new_order.assign_order_no()
    new_order["store_id"] = configs["store_id"]
    new_order["order_id"] = assigned_order_id
    new_order["serial_number"] = "u"+uber_display_id

    # Add order to database (order table)
    = add_to_table('order', new_order.tosplitted())

    # Turn this on if we want to notify local pos machine and TV.
    set_status_to_paid and send to websocket(new_order, **configs)
    indicator, message = True, "Order successfully added to the system"

    return {"indicator": indicator, "message": message, "orderid":
    new_order["order_id"]}
```

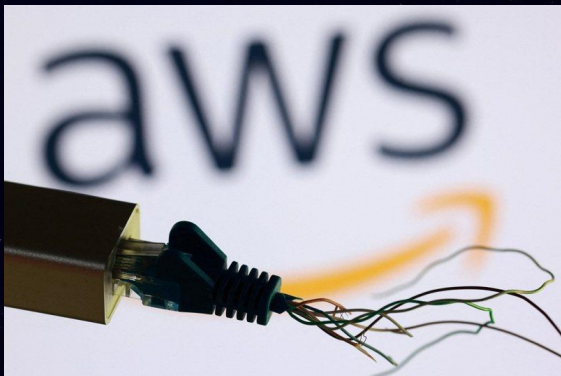
POS Machine



Challenges

1. Unpredictable consumer behaviors and edge cases across 4 distinct ordering methods.
2. Over the past 6 years, we have experienced AWS network outages, hardware jams, power loss, and physical cable damage (e.g., damage by mice).

Software must be fault-tolerant because the physical world is unpredictable.



Challenges

1. Unpredictable consumer behaviors and edge cases across 4 distinct ordering methods.

- Leveraging comprehensive logging and behavioral data collection to design a more inclusive and resilient system.

2. Over the past 6 years, we have experienced AWS network outages, hardware jams, power loss, and physical cable damage (e.g., damage by mice).

- **AWS network outages:** local/cloud design; if the cloud server fails, the local system can still function.
- **Hardware Jams:** modularize hardware parts; make it easy to replace when a part is broken.
- **Power Loss:** local/cloud server design; ensure data won't be lost when power is suddenly lost.
- **Physical Cable Damage:** modularize hardware replacement; system logs display the status of each hardware module.

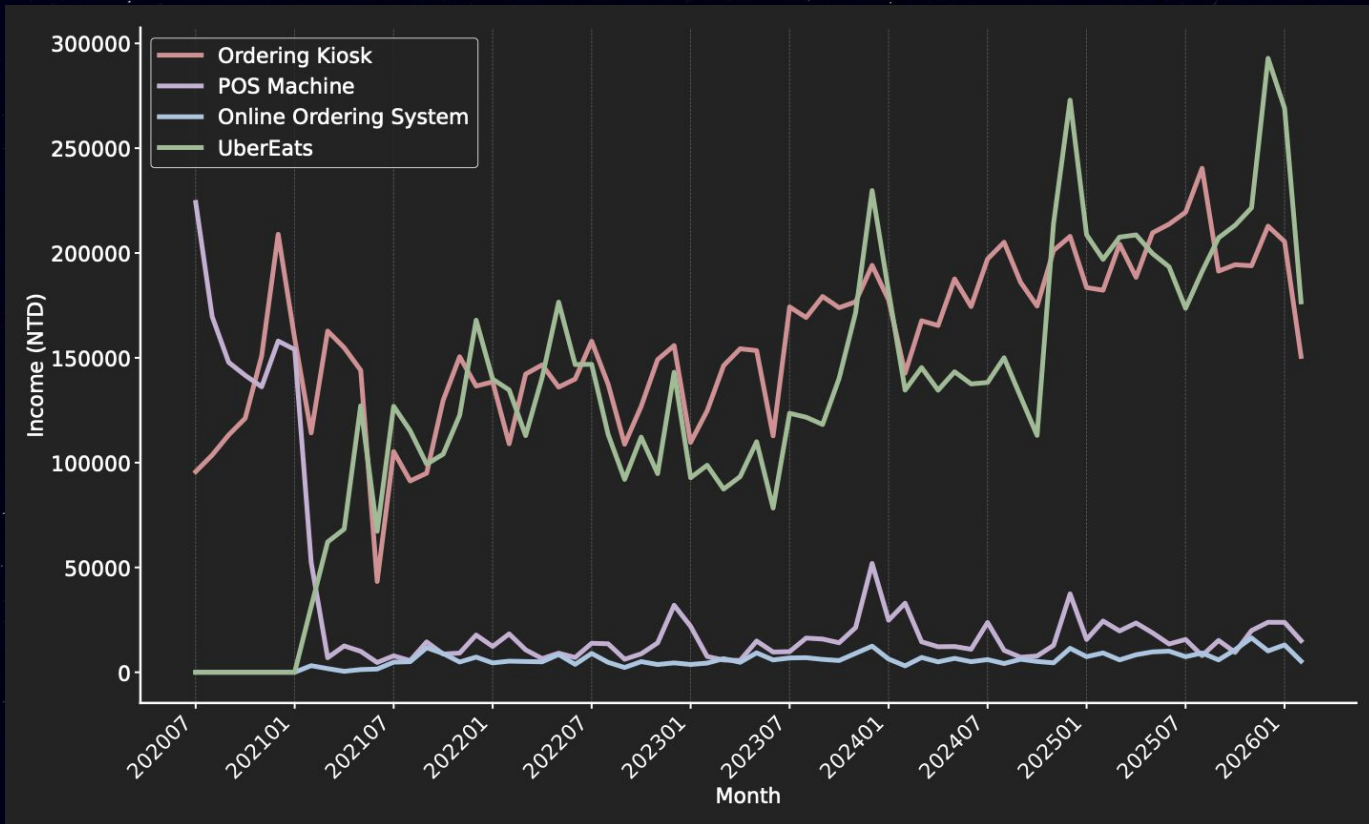
Results



Successfully supported the expansion of 3 retail stores over 6 years with highly stable system performance.



Results



Lessons Learned

- The real world is complicated; no matter how hard we try to include all possibilities in our system, there will always be some unexpected scenarios. How to make sure our system does not fail catastrophically is important.
- Automation is hard because humans are using it. If the world were just robots, it would be easy, but humans are unpredictable; we need to use these things to help humans without generating more problems.
- How to make a product rather than an academic project.

